Data and Artificial Intelligence
Cyber Shujaa Program

Week 9 Assignment
MLOPs

Student Name: Rodney Roy Gitonga
Student ID: CS-DA03-26025

Contents

Contents 1
Introduction 1
Tasks Completed 2
Step 1: Data Loading & Exploration 2
Step 2: Data Preparation 2
Step 3: Model Training and Hyperparameter Tuning 3
Step 4: Evaluation Results and Model Saving 3
Link to Code: 4
Conclusion 4

Introduction

The goal of this project is to build an end-to-end machine learning workflow to predict median house values (in $100,000s) using the California Housing dataset. This dataset features 1990 census data, including eight numerical features such as median income, house age, average rooms, and geographical coordinates.

The primary focus of this assignment is to move beyond simply training a model, and instead engineer a complete, deployable workflow. This involves structuring a data preprocessing Pipeline, utilizing ColumnTransformer for feature scaling and imputation, and applying GridSearchCV to systematically find the optimal hyperparameters for a KNeighborsRegressor model. The final model's predictive success is evaluated using the **R-squared (R^2)** score, alongside Mean Squared Error (MSE) and Root Mean Squared Error (RMSE).

Tasks Completed

Step 1: Data Loading & Exploration

I loaded the dataset directly via scikit-learn's fetch_california_housing utility. Because the end goal is a production-ready pipeline, I fetched the data as a pandas DataFrame for easier feature management and confirmed that it contained the expected 8 features and a target variable.

Rodney_Roy_Gitonga_MLOPs_CS_DA03_26025

File Edit View Insert Runtime Tools Help

```python
# Import libraries
import pandas as pd
import pickle
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import r2_score, mean_squared_error

# 1. Load dataset
print("Loading California Housing dataset...")
X, y = fetch_california_housing(return_X_y=True, as_frame=True)

# Explore the dataset
print(f"Dataset shape: {X.shape}")
display(X.head())
```

```
Loading California Housing dataset...
Dataset shape: (20640, 8)
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.25

Step 2: Data Preparation

I split the data into an 80% training set and a 20% test set. Because the k-Nearest Neighbors algorithm relies on calculating distances between data points, it is highly sensitive to the scale of the data. Features like 'Population' would completely overshadow features like 'MedInc' (Median Income) without scaling.

To handle this elegantly, I utilized a ColumnTransformer housing a nested Pipeline. This preprocessing pipeline automatically applies a SimpleImputer (to fill any potential missing values with the column mean) followed by a StandardScaler. This preprocessor was then chained directly to the KNeighborsRegressor.

```python
# 2. Train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Preprocessing: Imputation + Scaling for numerical features
numeric_features = X.columns  # All 8 features are numerical
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])

# 4. Combine preprocessing using ColumnTransformer
preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_transformer, numeric_features)
])

# 5. Build full pipeline: preprocessing + KNN Regressor
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('knn', KNeighborsRegressor())
])

print("Pipeline constructed successfully!")
```

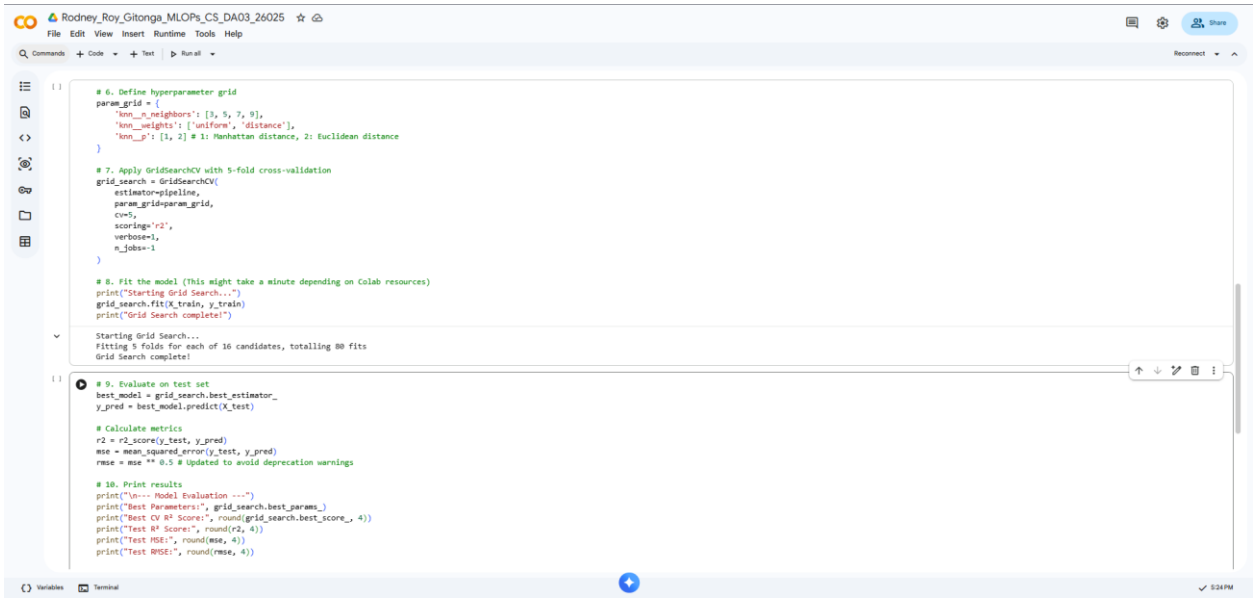
```
Pipeline constructed successfully!
```

Step 3: Model Training and Hyperparameter Tuning

Instead of guessing the best parameters for the KNN model, I utilized GridSearchCV with 5-fold cross-validation. I tested a grid of hyperparameter combinations:

- **n_neighbors:** 3, 5, 7, and 9
- **weights:** 'uniform' (all neighbors weighted equally) vs. 'distance' (closer neighbors have more influence)
- **p:** 1 (Manhattan distance) vs. 2 (Euclidean distance)

Training the model through the pipeline ensures that standard scaling is applied independently to each fold during cross-validation, strictly preventing data leakage from the validation folds into the training process.



```
# 6. Define hyperparameter grid
param_grid = {
    'knn__n_neighbors': [3, 5, 7, 9],
    'knn__weights': ['uniform', 'distance'],
    'knn__p': [1, 2] # 1: Manhattan distance, 2: Euclidean distance
}

# 7. Apply GridSearchCV with 5-fold cross-validation
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=param_grid,
    cv=5,
    scoring='r2',
    verbose=1,
    n_jobs=-1
)

# 8. Fit the model (This might take a minute depending on Colab resources)
print("Starting Grid Search...")
grid_search.fit(X_train, y_train)
print("Grid Search complete!")
```

```
Starting Grid Search...
Fitting 5 folds for each of 16 candidates, totalling 80 fits
Grid Search complete!
```

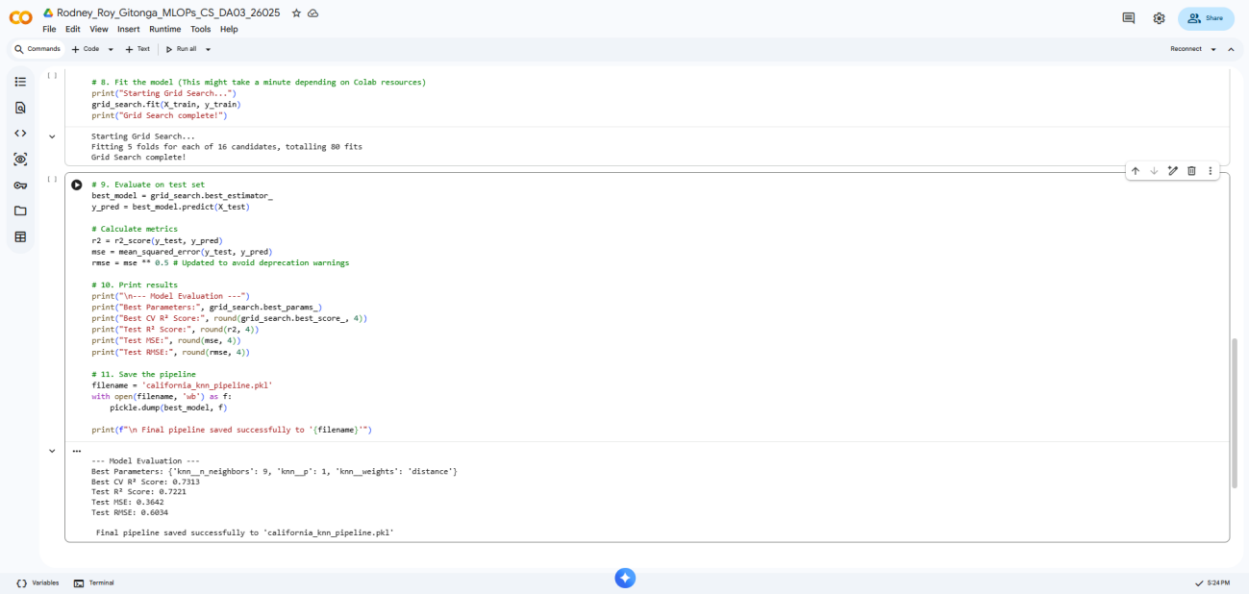
```
# 9. Evaluate on test set
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

# Calculate metrics
r2 = r2_score(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = mse ** 0.5 # Updated to avoid deprecation warnings

# 10. Print results
print("\n--- Model Evaluation ---")
print("Best Parameters:", grid_search.best_params_)
print("Best CV R² Score:", round(grid_search.best_score_, 4))
print("Test R² Score:", round(r2, 4))
print("Test MSE:", round(mse, 4))
print("Test RMSE:", round(rmse, 4))
```

Step 4: Evaluation Results and Model Saving

The Grid Search successfully identified the optimal hyperparameters for the housing data. The model was evaluated on the unseen 20% test set, yielding strong predictive performance. Finally, the entire trained pipeline (which includes both the scaler and the tuned model) was serialized and saved as a .pkl file using the pickle library, making it ready for deployment.



```
# 8. Fit the model (This might take a minute depending on Colab resources)
print("Starting Grid Search...")
grid_search.fit(X_train, y_train)
print("Grid Search complete!")
```

```
Starting Grid Search...
Fitting 5 folds for each of 16 candidates, totalling 80 fits
Grid Search complete!
```

```
# 9. Evaluate on test set
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

# Calculate metrics
r2 = r2_score(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = mse ** 0.5 # Updated to avoid deprecation warnings

# 10. Print results
print("\n--- Model Evaluation ---")
print("Best Parameters:", grid_search.best_params_)
print("Best CV R² Score:", round(grid_search.best_score_, 4))
print("Test R² Score:", round(r2, 4))
print("Test MSE:", round(mse, 4))
print("Test RMSE:", round(rmse, 4))

# 11. Save the pipeline
filename = 'california_knn_pipeline.pkl'
with open(filename, 'wb') as f:
    pickle.dump(best_model, f)

print(f"\n Final pipeline saved successfully to '{filename}'")
```

```
--- Model Evaluation ---
Best Parameters: {'knn__n_neighbors': 9, 'knn__p': 1, 'knn__weights': 'distance'}
Best CV R² Score: 0.7313
Test R² Score: 0.7221
Test MSE: 0.3642
Test RMSE: 0.6034

 Final pipeline saved successfully to 'california_knn_pipeline.pkl'
```

Link to Code:

https://colab.research.google.com/drive/1g7R3DPOvgKXPO8-zE9VvYrpDA69TMC0R?usp=sharing

Conclusion

This assignment effectively demonstrated the architecture of a professional machine learning workflow. By wrapping the preprocessing steps (imputation and scaling) and the estimator (KNN) into a single scikit-learn Pipeline, the code became much cleaner and significantly less prone to errors like data leakage during cross-validation.

I learned that hyperparameter tuning via GridSearchCV is a powerful, systematic way to maximize a model's performance, as it exhaustively tests combinations (like switching between Euclidean and Manhattan distances) to find the best fit for the specific geometry of the dataset. Finally, saving the entire pipeline as a .pkl object highlighted how seamlessly a model can be transitioned from a research notebook environment into a deployable state, ready to ingest and predict on new, raw JSON data in a web application.